

12_forward_hybrid_mlp_distance_test

April 28, 2026

1 12 - Forward Hybrid MLP Distance Test

This notebook tests a **target-specific hybrid strategy** after the diagnostics notebook:

- Keep the validated final ExtraTrees pipeline for fragmentation targets: P80, fines_frac, oversize_frac.
- Test whether MLP can improve distance targets: R95, R50_fines, R50_oversize.
- Test weighted blends between ExtraTrees and MLP for distance targets only.

The notebook **does not overwrite** the official prediction_submission.csv unless explicitly enabled.

Main question:

Can we improve distance targets without damaging fragmentation targets or the inverse-design feasibility metrics?

1.1 1. Imports and paths

```
[1]: from pathlib import Path
import json
import warnings
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 180)
```

```

PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.
    cwd()
FORWARD_DIR = PROJECT_ROOT / "data" / "raw" / "forward_prediction"
INVERSE_DIR = PROJECT_ROOT / "data" / "raw" / "inverse_design"
OUTPUTS_DIR = PROJECT_ROOT / "outputs"
SUBMISSIONS_DIR = OUTPUTS_DIR / "submissions"
MODELS_DIR = OUTPUTS_DIR / "models"
REPORTS_DIR = PROJECT_ROOT / "reports"
FIGURES_DIR = OUTPUTS_DIR / "figures"

for p in [SUBMISSIONS_DIR, MODELS_DIR, REPORTS_DIR, FIGURES_DIR]:
    p.mkdir(parents=True, exist_ok=True)

print("Project root:", PROJECT_ROOT)
print("Forward dir:", FORWARD_DIR)
print("Submissions dir:", SUBMISSIONS_DIR)
print("Reports dir:", REPORTS_DIR)

```

Project root: /home/alouiyaz/projects/boom-challenge-ejecta-prediction
Forward dir: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/data/raw/forward_prediction
Submissions dir: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/submissions
Reports dir: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/reports

1.2 2. Load data and constraints

```

[2]: input_cols = [
    "energy", "angle_rad", "coupling", "strength",
    "porosity", "gravity", "atmosphere", "shape_factor",
]

target_cols = [
    "P80", "fines_frac", "oversize_frac",
    "R95", "R50_fines", "R50_oversize",
]

fragmentation_targets = ["P80", "fines_frac", "oversize_frac"]
distance_targets = ["R95", "R50_fines", "R50_oversize"]

raw_train = pd.read_csv(FORWARD_DIR / "train.csv")[input_cols]
raw_test = pd.read_csv(FORWARD_DIR / "test.csv")[input_cols]
y = pd.read_csv(FORWARD_DIR / "train_labels.csv")[target_cols]

constraints_path = INVERSE_DIR / "constraints.json"
if constraints_path.exists():
    with open(constraints_path, "r") as f:
        constraints_data = json.load(f)

```

```

        output_constraints = constraints_data["constraints"]
    else:
        output_constraints = {"p80_min": 96.0, "p80_max": 101.0, "r95_max": 175.0}

p80_min = output_constraints["p80_min"]
p80_max = output_constraints["p80_max"]
r95_max = output_constraints["r95_max"]

print("Raw train:", raw_train.shape)
print("Raw test:", raw_test.shape)
print("Targets:", y.shape)
display(pd.DataFrame([output_constraints]))

```

Raw train: (2930, 8)

Raw test: (492, 8)

Targets: (2930, 6)

	p80_min	p80_max	r95_max
0	96.0	101.0	175.0

1.3 3. Feature engineering and final ExtraTrees configuration

This reproduces the final forward model logic from notebook 09.

```

[3]: class PhysicsFeatureEngineer(BaseEstimator, TransformerMixin):
    def __init__(self, use_advanced: bool = False):
        self.use_advanced = use_advanced

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = pd.DataFrame(X).copy()
        X = X[input_cols].copy()
        eps = 1e-9

        X["effective_energy"] = X["energy"] * X["coupling"]
        X["log_energy"] = np.log1p(X["energy"])
        X["log_effective_energy"] = np.log1p(X["effective_energy"])

        X["sin_angle"] = np.sin(X["angle_rad"])
        X["cos_angle"] = np.cos(X["angle_rad"])
        X["tan_angle"] = np.tan(X["angle_rad"])
        X["horizontal_energy"] = X["effective_energy"] * X["cos_angle"]
        X["vertical_energy"] = X["effective_energy"] * X["sin_angle"]
        X["vertical_horizontal_ratio"] = X["vertical_energy"] /
        ↪(X["horizontal_energy"] + eps)

```

```

X["energy_per_strength"] = X["energy"] / (X["strength"] + eps)
X["effective_energy_per_strength"] = X["effective_energy"] /
↪(X["strength"] + eps)
X["material_resistance_index"] = X["strength"] * (1 - X["porosity"])
X["fragmentation_index"] = X["effective_energy"] * X["porosity"] /
↪(X["strength"] + eps)
X["coupling_porosity"] = X["coupling"] * X["porosity"]
X["coupling_atmosphere"] = X["coupling"] * X["atmosphere"]
X["porosity_strength"] = X["porosity"] * X["strength"]

X["energy_per_gravity"] = X["energy"] / (X["gravity"] + eps)
X["effective_energy_per_gravity"] = X["effective_energy"] /
↪(X["gravity"] + eps)
X["horizontal_energy_per_gravity"] = X["horizontal_energy"] /
↪(X["gravity"] + eps)
X["vertical_energy_per_gravity"] = X["vertical_energy"] / (X["gravity"]
↪+ eps)

X["drag_proxy"] = X["atmosphere"] * X["shape_factor"]
X["drag_per_gravity"] = X["drag_proxy"] / (X["gravity"] + eps)
X["atmosphere_shape_energy"] = X["atmosphere"] * X["shape_factor"] *
↪X["effective_energy"]
X["atmosphere_per_gravity"] = X["atmosphere"] / (X["gravity"] + eps)

X["pi_gravity_proxy"] = (X["gravity"] * X["coupling"]) / (X["energy"] +
↪eps)
X["pi_strength_proxy"] = X["strength"] / (X["gravity"] * X["coupling"]
↪+ eps)
X["pi_atmosphere_proxy"] = X["atmosphere"] / (X["gravity"] *
↪X["coupling"] + eps)

X["porosity_regime"] = (X["porosity"] > 0.15).astype(int)
X["strength_regime"] = (X["strength"] > 2.6).astype(int)
X["angle_regime"] = (X["angle_rad"] > 0.95).astype(int)
X["atm_regime"] = (X["atmosphere"] > 0.30).astype(int)
X["regime_combo"] = (
    X["porosity_regime"] * 8
    + X["strength_regime"] * 4
    + X["angle_regime"] * 2
    + X["atm_regime"]
)

X["scaled_energy"] = X["effective_energy"] / (X["strength"] * np.
↪sqrt(X["gravity"] + eps)
X["fragility"] = X["porosity"] / (X["strength"] + eps)

```

```

        X["range_proxy"] = (X["effective_energy"] * (X["cos_angle"] ** 2)) / (
↪(X["gravity"] * X["strength"] + eps)
        X["energy_sin_angle"] = X["energy"] * X["sin_angle"]
        X["momentum_proxy"] = X["effective_energy"] * X["sin_angle"]
        X["coupling_per_atm_clipped"] = X["coupling"] / (X["atmosphere"] + 1e-3)
        X["log_coupling_per_atm"] = np.log1p(X["coupling_per_atm_clipped"])
        X["retention_factor"] = X["atmosphere"] * X["drag_proxy"] / (
↪(X["energy"] + eps)

        if self.use_advanced:
            X["froude_proxy"] = np.sqrt(X["effective_energy"] / (X["gravity"] +
↪eps))

            X["stress_ratio_compact"] = X["effective_energy"] / (
↪(X["material_resistance_index"] + eps)
            X["sqrt_effective_energy_per_strength"] = np.
↪sqrt(X["effective_energy_per_strength"].clip(lower=0))
            X["sqrt_effective_energy_per_gravity"] = np.
↪sqrt(X["effective_energy_per_gravity"].clip(lower=0))

        return X

class ColumnSelector(BaseEstimator, TransformerMixin):
    def __init__(self, columns):
        self.columns = list(columns)

    def fit(self, X, y=None):
        missing = [c for c in self.columns if c not in X.columns]
        if missing:
            raise ValueError(f"Missing columns: {missing}")
        return self

    def transform(self, X):
        return X[self.columns].copy()

raw_features = input_cols.copy()
fragmentation_features_v1 = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "drag_proxy", "atmosphere_shape_energy",
    "pi_strength_proxy", "pi_atmosphere_proxy", "scaled_energy", "fragility",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
↪"regime_combo",

```

```

]
fragmentation_features_v2 = fragmentation_features_v1 + [
    "stress_ratio_compact",
    "sqrt_effective_energy_per_strength",
]
distance_features_v1 = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy",
    ↪ "atmosphere_per_gravity",
    "pi_gravity_proxy", "pi_atmosphere_proxy", "scaled_energy", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↪ "regime_combo",
]

final_target_config = {
    "P80": {"feature_version": "v2", "features": fragmentation_features_v2},
    "fines_frac": {"feature_version": "v2", "features":
    ↪ fragmentation_features_v2},
    "oversize_frac": {"feature_version": "v2", "features":
    ↪ fragmentation_features_v2},
    "R95": {"feature_version": "v1", "features": distance_features_v1},
    "R50_fines": {"feature_version": "v1", "features": distance_features_v1},
    "R50_oversize": {"feature_version": "v1", "features": distance_features_v1},
}

display(pd.DataFrame([
    {"target": t, "feature_version": cfg["feature_version"], "n_features":
    ↪ len(cfg["features"])}
    for t, cfg in final_target_config.items()
]))

```

	target	feature_version	n_features
0	P80	v2	36
1	fines_frac	v2	36
2	oversize_frac	v2	36
3	R95	v1	33
4	R50_fines	v1	33
5	R50_oversize	v1	33

1.4 4. Model builders

```
[4]: def build_extratrees(random_state=42, n_estimators=800):
    return ExtraTreesRegressor(
        n_estimators=n_estimators,
        max_features="sqrt",
        min_samples_leaf=2,
        random_state=random_state,
        n_jobs=-1,
    )

def build_target_et_pipeline(target, random_state=42, n_estimators=800):
    cfg = final_target_config[target]
    use_advanced = cfg["feature_version"] == "v2"
    return Pipeline(steps=[
        ("features", PhysicsFeatureEngineer(use_advanced=use_advanced)),
        ("select", ColumnSelector(cfg["features"])),
        ("model", build_extratrees(random_state=random_state,
    ↪n_estimators=n_estimators)),
    ])

def build_mlp(random_state=42):
    return Pipeline(steps=[
        ("scaler", StandardScaler()),
        ("mlp", MLPRegressor(
            hidden_layer_sizes=(128, 64),
            activation="relu",
            solver="adam",
            alpha=1e-3,
            learning_rate_init=1e-3,
            max_iter=1200,
            early_stopping=True,
            validation_fraction=0.15,
            n_iter_no_change=40,
            random_state=random_state,
        )),
    ])

def build_feature_matrix_for_target(X_raw, target):
    cfg = final_target_config[target]
    X_fe = PhysicsFeatureEngineer(use_advanced=(cfg["feature_version"] ==
    ↪"v2")).fit_transform(X_raw)
    return X_fe[cfg["features"]].copy()

def clip_predictions(preds, target):
    preds = np.asarray(preds).copy()
    if target in ["fines_frac", "oversize_frac"]:
```

```

    return np.clip(preds, 0, 1)
return np.clip(preds, 0, None)

```

1.5 5. Metrics helpers

```

[5]: def regression_metrics(y_true, y_pred, target_name=None):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    abs_error = np.abs(y_pred - y_true)
    out = {
        "MAE": mean_absolute_error(y_true, y_pred),
        "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
        "R2": r2_score(y_true, y_pred),
        "Median_AE": np.median(abs_error),
        "P90_AE": np.percentile(abs_error, 90),
        "P95_AE": np.percentile(abs_error, 95),
        "Max_AE": np.max(abs_error),
        "Bias": float(np.mean(y_pred - y_true)),
    }
    if target_name is not None:
        out["normalized_MAE"] = out["MAE"] / (y[target_name].std() + 1e-9)
        out["normalized_RMSE"] = out["RMSE"] / (y[target_name].std() + 1e-9)
    return out

def feasibility_mask(df_targets):
    return (df_targets["P80"].between(p80_min, p80_max)) & (df_targets["R95"]_
    ↪ <= r95_max)

def custom_constraint_metrics(y_true_df, y_pred_df):
    true_feasible = feasibility_mask(y_true_df)
    pred_feasible = feasibility_mask(y_pred_df)
    tp = int((true_feasible & pred_feasible).sum())
    fp = int((~true_feasible & pred_feasible).sum())
    fn = int((true_feasible & ~pred_feasible).sum())
    precision = tp / (tp + fp + 1e-9)
    recall = tp / (tp + fn + 1e-9)
    f1 = 2 * precision * recall / (precision + recall + 1e-9)
    near_zone = (y_true_df["P80"].between(80, 120)) & (y_true_df["R95"] <= 250)
    out = {
        "n_true_feasible": int(true_feasible.sum()),
        "n_pred_feasible": int(pred_feasible.sum()),
        "true_positive": tp,
        "false_positive": fp,
        "false_negative": fn,
        "feasible_precision": precision,
        "feasible_recall": recall,
        "feasible_f1": f1,
    }

```



```

        "near_zone_count": int(near_zone.sum()),
    }
    if near_zone.sum() > 0:
        out["near_zone_MAE_P80"] = mean_absolute_error(y_true_df.loc[near_zone,
↪ "P80"], y_pred_df.loc[near_zone, "P80"])
        out["near_zone_MAE_R95"] = mean_absolute_error(y_true_df.loc[near_zone,
↪ "R95"], y_pred_df.loc[near_zone, "R95"])
        out["near_zone_R95_bias"] = float((y_pred_df.loc[near_zone, "R95"] -
↪ y_true_df.loc[near_zone, "R95"]).mean())
    return out

def evaluate_prediction_frame(pred_df, strategy_name):
    rows = []
    for target in target_cols:
        m = regression_metrics(y[target], pred_df[target], target_name=target)
        m["target"] = target
        m["strategy"] = strategy_name
        rows.append(m)
    return pd.DataFrame(rows), pd.DataFrame([{"**custom_constraint_metrics(y,
↪ pred_df), "strategy": strategy_name}])

```

1.6 6. Cross-validated OOF predictions for ExtraTrees and MLP

We generate OOF predictions for:

- ExtraTrees final model for all targets.
- MLP for distance targets only.

Fragmentation targets remain ExtraTrees in all hybrid candidates.

```

[6]: def cross_validate_et_and_mlp_distance(X_raw, y_df, n_splits=5,
↪ random_state=42, n_estimators=800):
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    et_oof = pd.DataFrame(index=y_df.index, columns=target_cols, dtype=float)
    mlp_distance_oof = pd.DataFrame(index=y_df.index, columns=distance_targets,
↪ dtype=float)

    for fold, (tr_idx, va_idx) in enumerate(kf.split(X_raw), start=1):
        print(f"Fold {fold}/{n_splits}")
        X_tr_raw = X_raw.iloc[tr_idx].reset_index(drop=True)
        X_va_raw = X_raw.iloc[va_idx].reset_index(drop=True)
        y_tr = y_df.iloc[tr_idx].reset_index(drop=True)

        # ExtraTrees for all targets
        for i, target in enumerate(target_cols):
            et_pipe = build_target_et_pipeline(target,
↪ random_state=random_state + fold * 10 + i, n_estimators=n_estimators)
            et_pipe.fit(X_tr_raw, y_tr[target])

```

```

        pred = et_pipe.predict(X_va_raw)
        et_oof.loc[va_idx, target] = clip_predictions(pred, target)

        # MLP only for distance targets
        for i, target in enumerate(distance_targets):
            X_tr_target = build_feature_matrix_for_target(X_tr_raw, target)
            X_va_target = build_feature_matrix_for_target(X_va_raw, target)
            mlp = build_mlp(random_state=random_state + fold * 100 + i)
            mlp.fit(X_tr_target, y_tr[target])
            pred = mlp.predict(X_va_target)
            mlp_distance_oof.loc[va_idx, target] = clip_predictions(pred,
↪target)

        return et_oof[target_cols], mlp_distance_oof[distance_targets]

RUN_HYBRID_DISTANCE_CV = True
if RUN_HYBRID_DISTANCE_CV:
    et_oof, mlp_distance_oof = cross_validate_et_and_mlp_distance(
        raw_train, y, n_splits=5, random_state=42, n_estimators=800
    )

    et_metrics, et_constraint = evaluate_prediction_frame(et_oof,
↪"ExtraTrees_final")

    # MLP-only distance strategy: fragmentation from ET, distances from MLP
    mlp_distance_strategy = et_oof.copy()
    for target in distance_targets:
        mlp_distance_strategy[target] = mlp_distance_oof[target]
    mlp_dist_metrics, mlp_dist_constraint =
↪evaluate_prediction_frame(mlp_distance_strategy, "ET_frag_MLP_dist")

    display(et_metrics.sort_values("normalized_MAE"))
    display(et_constraint)
    display(mlp_dist_metrics.sort_values("normalized_MAE"))
    display(mlp_dist_constraint)
else:
    print("Skipped CV.")

```

Fold 1/5
 Fold 2/5
 Fold 3/5
 Fold 4/5
 Fold 5/5

	MAE	RMSE	R2	Median_AE	P90_AE	P95_AE	Max_AE
↪	Bias	normalized_MAE	normalized_RMSE	target	strategy		
2	0.025688	0.035234	0.990528	0.018906	0.057539	0.074997	0.168759
↪	-0.000083	0.070944	0.097309	oversize_frac	ExtraTrees_final		

```

1  0.006320  0.013829  0.959351  0.000615  0.019699  0.031068  0.140887
↪-0.000029      0.092129      0.201582      fines_frac  ExtraTrees_final
0  7.653693  10.163284  0.976102  5.938123  16.682502  21.564071  53.147350
↪-0.003650      0.116397      0.154562      P80  ExtraTrees_final
3  41.418662  68.567830  0.917348  20.866800  104.336350  151.864970  466.861680
↪-0.447065      0.173632      0.287444      R95  ExtraTrees_final
4  50.379136  78.486939  0.898911  27.273152  130.409337  172.456100  535.118662
↪-0.456241      0.204048      0.317891      R50_fines  ExtraTrees_final
5  22.757123  38.642620  0.872964  11.697269  56.942342  81.415772  351.395369
↪ 0.007746      0.209865      0.356360      R50_oversize  ExtraTrees_final

n_true_feasible  n_pred_feasible  true_positive  false_positive
↪false_negative  feasible_precision  feasible_recall  feasible_f1
↪near_zone_count  near_zone_MAE_P80  \
0      35      37      12      25
↪23      0.324324      0.342857      0.333333      372
↪ 4.591746

near_zone_MAE_R95  near_zone_R95_bias  strategy
0      25.543196      13.99367  ExtraTrees_final

MAE      RMSE      R2  Median_AE      P90_AE      P95_AE      Max_AE
↪ Bias  normalized_MAE  normalized_RMSE      target      strategy
2  0.025688  0.035234  0.990528  0.018906  0.057539  0.074997  0.168759
↪-0.000083      0.070944      0.097309  oversize_frac  ET_frag_MLP_dist
1  0.006320  0.013829  0.959351  0.000615  0.019699  0.031068  0.140887
↪-0.000029      0.092129      0.201582      fines_frac  ET_frag_MLP_dist
0  7.653693  10.163284  0.976102  5.938123  16.682502  21.564071  53.147350
↪-0.003650      0.116397      0.154562      P80  ET_frag_MLP_dist
3  40.700694  66.210892  0.922932  22.160519  100.292317  146.500041  522.469419
↪ 0.115849      0.170622      0.277564      R95  ET_frag_MLP_dist
4  49.800116  77.178054  0.902254  28.090191  129.192209  170.243980  517.555006
↪-0.508429      0.201703      0.312590      R50_fines  ET_frag_MLP_dist
5  22.495511  37.862863  0.878039  11.976442  54.911018  79.048671  312.284013
↪-0.314844      0.207452      0.349169      R50_oversize  ET_frag_MLP_dist

n_true_feasible  n_pred_feasible  true_positive  false_positive
↪false_negative  feasible_precision  feasible_recall  feasible_f1
↪near_zone_count  near_zone_MAE_P80  \
0      35      37      12      25
↪23      0.324324      0.342857      0.333333      372
↪ 4.591746

near_zone_MAE_R95  near_zone_R95_bias  strategy
0      24.579639      13.247459  ET_frag_MLP_dist

```

1.7 7. Weighted blends for distance targets

We keep fragmentation targets from ExtraTrees and blend distance targets:

$\text{distance_pred} = (1 - w) * \text{ExtraTrees} + w * \text{MLP}$

where w is the MLP weight.

```
[7]: if RUN_HYBRID_DISTANCE_CV:
    blend_weights = [0.0, 0.15, 0.25, 0.35, 0.50, 0.65, 0.75, 1.0]
    blend_metric_tables = []
    blend_constraint_tables = []
    blend_predictions = {}

    for w in blend_weights:
        pred = et_oof.copy()
        for target in distance_targets:
            pred[target] = (1 - w) * et_oof[target] + w * mlp_distance_oof[target]
            pred[target] = clip_predictions(pred[target], target)

        strategy = f"blend_distance_mlp_w_{w:.2f}"
        metrics_df, constraint_df = evaluate_prediction_frame(pred, strategy)
        metrics_df["mlp_weight"] = w
        constraint_df["mlp_weight"] = w
        blend_metric_tables.append(metrics_df)
        blend_constraint_tables.append(constraint_df)
        blend_predictions[w] = pred

    blend_metrics = pd.concat(blend_metric_tables, ignore_index=True)
    blend_constraints = pd.concat(blend_constraint_tables, ignore_index=True)

    display(blend_metrics.sort_values(["target", "MAE"])[[
        "strategy", "mlp_weight", "target", "MAE", "RMSE", "R2", "P95_AE", "normalized_MAE", "normalized_RMSE"
    ]])
    display(blend_constraints.sort_values("feasible_f1", ascending=False))
else:
    print("Run CV first.")
```

	strategy	mlp_weight	target	MAE	RMSE
↪	R2	P95_AE	normalized_MAE	normalized_RMSE	
0	blend_distance_mlp_w_0.00	0.00	P80	7.653693	10.163284
↪	0.976102	21.564071	0.116397	0.154562	
6	blend_distance_mlp_w_0.15	0.15	P80	7.653693	10.163284
↪	0.976102	21.564071	0.116397	0.154562	
12	blend_distance_mlp_w_0.25	0.25	P80	7.653693	10.163284
↪	0.976102	21.564071	0.116397	0.154562	

18	blend_distance_mlp_w_0.35	0.35	P80	7.653693	10.163284	┐
	└0.976102 21.564071	0.116397	0.154562			
24	blend_distance_mlp_w_0.50	0.50	P80	7.653693	10.163284	┐
	└0.976102 21.564071	0.116397	0.154562			
30	blend_distance_mlp_w_0.65	0.65	P80	7.653693	10.163284	┐
	└0.976102 21.564071	0.116397	0.154562			
36	blend_distance_mlp_w_0.75	0.75	P80	7.653693	10.163284	┐
	└0.976102 21.564071	0.116397	0.154562			
42	blend_distance_mlp_w_1.00	1.00	P80	7.653693	10.163284	┐
	└0.976102 21.564071	0.116397	0.154562			
34	blend_distance_mlp_w_0.65	0.65	R50_fines	49.508153	76.997295	┐
	└0.902712 170.148148	0.200520	0.311858			
40	blend_distance_mlp_w_0.75	0.75	R50_fines	49.530768	76.978224	┐
	└0.902760 168.453954	0.200612	0.311781			
28	blend_distance_mlp_w_0.50	0.50	R50_fines	49.553594	77.131942	┐
	└0.902371 171.729602	0.200704	0.312403			
22	blend_distance_mlp_w_0.35	0.35	R50_fines	49.678018	77.393202	┐
	└0.901708 171.156764	0.201208	0.313461			
46	blend_distance_mlp_w_1.00	1.00	R50_fines	49.800116	77.178054	┐
	└0.902254 170.243980	0.201703	0.312590			
16	blend_distance_mlp_w_0.25	0.25	R50_fines	49.823281	77.637109	┐
	└0.901088 171.467200	0.201796	0.314449			
10	blend_distance_mlp_w_0.15	0.15	R50_fines	50.017477	77.936211	┐
	└0.900324 172.313445	0.202583	0.315661			
4	blend_distance_mlp_w_0.00	0.00	R50_fines	50.379136	78.486939	┐
	└0.898911 172.456100	0.204048	0.317891			
35	blend_distance_mlp_w_0.65	0.65	R50_oversize	22.317600	37.833312	┐
	└0.878230 80.327353	0.205812	0.348897			
29	blend_distance_mlp_w_0.50	0.50	R50_oversize	22.328939	37.921256	┐
	└0.877663 81.913925	0.205916	0.349708			
41	blend_distance_mlp_w_0.75	0.75	R50_oversize	22.339935	37.808178	┐
	└0.878391 79.229972	0.206018	0.348665			
23	blend_distance_mlp_w_0.35	0.35	R50_oversize	22.399170	38.069098	┐
	└0.876707 81.275054	0.206564	0.351071			
17	blend_distance_mlp_w_0.25	0.25	R50_oversize	22.474039	38.200605	┐
	└0.875854 81.909575	0.207254	0.352284			
47	blend_distance_mlp_w_1.00	1.00	R50_oversize	22.495511	37.862863	┐
	└0.878039 79.048671	0.207452	0.349169			
11	blend_distance_mlp_w_0.15	0.15	R50_oversize	22.567222	38.358160	┐
	└0.874828 81.737928	0.208114	0.353737			
5	blend_distance_mlp_w_0.00	0.00	R50_oversize	22.757123	38.642620	┐
	└0.872964 81.415772	0.209865	0.356360			
33	blend_distance_mlp_w_0.65	0.65	R95	40.236457	65.973277	┐
	└0.923484 145.924860	0.168676	0.276568			
27	blend_distance_mlp_w_0.50	0.50	R95	40.285680	66.227038	┐
	└0.922895 146.924912	0.168882	0.277631			

39	blend_distance_mlp_w_0.75	0.75	R95	40.289032	65.922514	⌵
	↪0.923602 145.389447	0.168896	0.276355			
21	blend_distance_mlp_w_0.35	0.35	R95	40.463501	66.691625	⌵
	↪0.921809 147.832806	0.169628	0.279579			
15	blend_distance_mlp_w_0.25	0.25	R95	40.643057	67.116394	⌵
	↪0.920810 148.610238	0.170380	0.281360			
45	blend_distance_mlp_w_1.00	1.00	R95	40.700694	66.210892	⌵
	↪0.922932 146.500041	0.170622	0.277564			
9	blend_distance_mlp_w_0.15	0.15	R95	40.897175	67.631232	⌵
	↪0.919590 149.640300	0.171446	0.283518			
3	blend_distance_mlp_w_0.00	0.00	R95	41.418662	68.567830	⌵
	↪0.917348 151.864970	0.173632	0.287444			
1	blend_distance_mlp_w_0.00	0.00	finest_frac	0.006320	0.013829	⌵
	↪0.959351 0.031068	0.092129	0.201582			
7	blend_distance_mlp_w_0.15	0.15	finest_frac	0.006320	0.013829	⌵
	↪0.959351 0.031068	0.092129	0.201582			
13	blend_distance_mlp_w_0.25	0.25	finest_frac	0.006320	0.013829	⌵
	↪0.959351 0.031068	0.092129	0.201582			
19	blend_distance_mlp_w_0.35	0.35	finest_frac	0.006320	0.013829	⌵
	↪0.959351 0.031068	0.092129	0.201582			
25	blend_distance_mlp_w_0.50	0.50	finest_frac	0.006320	0.013829	⌵
	↪0.959351 0.031068	0.092129	0.201582			
31	blend_distance_mlp_w_0.65	0.65	finest_frac	0.006320	0.013829	⌵
	↪0.959351 0.031068	0.092129	0.201582			
37	blend_distance_mlp_w_0.75	0.75	finest_frac	0.006320	0.013829	⌵
	↪0.959351 0.031068	0.092129	0.201582			
43	blend_distance_mlp_w_1.00	1.00	finest_frac	0.006320	0.013829	⌵
	↪0.959351 0.031068	0.092129	0.201582			
2	blend_distance_mlp_w_0.00	0.00	oversize_frac	0.025688	0.035234	⌵
	↪0.990528 0.074997	0.070944	0.097309			
8	blend_distance_mlp_w_0.15	0.15	oversize_frac	0.025688	0.035234	⌵
	↪0.990528 0.074997	0.070944	0.097309			
14	blend_distance_mlp_w_0.25	0.25	oversize_frac	0.025688	0.035234	⌵
	↪0.990528 0.074997	0.070944	0.097309			
20	blend_distance_mlp_w_0.35	0.35	oversize_frac	0.025688	0.035234	⌵
	↪0.990528 0.074997	0.070944	0.097309			
26	blend_distance_mlp_w_0.50	0.50	oversize_frac	0.025688	0.035234	⌵
	↪0.990528 0.074997	0.070944	0.097309			
32	blend_distance_mlp_w_0.65	0.65	oversize_frac	0.025688	0.035234	⌵
	↪0.990528 0.074997	0.070944	0.097309			
38	blend_distance_mlp_w_0.75	0.75	oversize_frac	0.025688	0.035234	⌵
	↪0.990528 0.074997	0.070944	0.097309			
44	blend_distance_mlp_w_1.00	1.00	oversize_frac	0.025688	0.035234	⌵
	↪0.990528 0.074997	0.070944	0.097309			

	n_true_feasible	n_pred_feasible	true_positive	false_positive	↵false_negative	feasible_precision	feasible_recall	feasible_f1	↵
	↵near_zone_count	near_zone_MAE_P80	\						
0	35	37	12	25					↵
↵23	0.324324	0.342857	0.333333	372					↵
↵4.591746									
1	35	37	12	25					↵
↵23	0.324324	0.342857	0.333333	372					↵
↵4.591746									
2	35	37	12	25					↵
↵23	0.324324	0.342857	0.333333	372					↵
↵4.591746									
3	35	37	12	25					↵
↵23	0.324324	0.342857	0.333333	372					↵
↵4.591746									
4	35	37	12	25					↵
↵23	0.324324	0.342857	0.333333	372					↵
↵4.591746									
5	35	37	12	25					↵
↵23	0.324324	0.342857	0.333333	372					↵
↵4.591746									
6	35	37	12	25					↵
↵23	0.324324	0.342857	0.333333	372					↵
↵4.591746									
7	35	37	12	25					↵
↵23	0.324324	0.342857	0.333333	372					↵
↵4.591746									

	near_zone_MAE_R95	near_zone_R95_bias	strategy	mlp_weight
0	25.543196	13.993670	blend_distance_mlp_w_0.00	0.00
1	25.092594	13.881738	blend_distance_mlp_w_0.15	0.15
2	24.827579	13.807117	blend_distance_mlp_w_0.25	0.25
3	24.619977	13.732496	blend_distance_mlp_w_0.35	0.35
4	24.403969	13.620565	blend_distance_mlp_w_0.50	0.50
5	24.343948	13.508633	blend_distance_mlp_w_0.65	0.65
6	24.342778	13.434012	blend_distance_mlp_w_0.75	0.75
7	24.579639	13.247459	blend_distance_mlp_w_1.00	1.00

1.8 8. Strategy-level selection

The selection should not rely only on mean normalized MAE. We also inspect:

- mean normalized RMSE;
- P95 error;
- feasible F1;
- near-zone R95 MAE.

```

[8]: if RUN_HYBRID_DISTANCE_CV:
    strategy_summary = blend_metrics.groupby(["strategy", "mlp_weight"]).agg(
        mean_normalized_MAE=("normalized_MAE", "mean"),
        mean_normalized_RMSE=("normalized_RMSE", "mean"),
        mean_R2=("R2", "mean"),
        mean_P95_AE=("P95_AE", "mean"),
    ).reset_index()

    strategy_summary = strategy_summary.merge(
        blend_constraints[[
            "strategy", "mlp_weight", "feasible_precision", "feasible_recall",
            ↪ "feasible_f1",
            "near_zone_MAE_P80", "near_zone_MAE_R95", "near_zone_R95_bias"
        ]],
        on=["strategy", "mlp_weight"],
        how="left",
    )

    display(strategy_summary.sort_values("mean_normalized_MAE"))

    # Conservative decision score: normalized MAE + normalized RMSE + small
    ↪penalty for lower feasibility F1.
    max_f1 = strategy_summary["feasible_f1"].max()
    strategy_summary["decision_score"] = (
        strategy_summary["mean_normalized_MAE"]
        + 0.40 * strategy_summary["mean_normalized_RMSE"]
        + 0.05 * (max_f1 - strategy_summary["feasible_f1"])
    )

    display(strategy_summary.sort_values("decision_score"))

    best_row = strategy_summary.sort_values("decision_score").iloc[0]
    BEST_MLP_WEIGHT = float(best_row["mlp_weight"])
    print("Best MLP weight by conservative decision score:", BEST_MLP_WEIGHT)
else:
    BEST_MLP_WEIGHT = 0.0
    print("CV skipped. Default BEST_MLP_WEIGHT = 0.0")

```

	strategy	mlp_weight	mean_normalized_MAE	mean_normalized_RMSE	mean_R2	mean_P95_AE	feasible_precision	feasible_recall	feasible_f1	near_zone_MAE_P80	\
5	blend_distance_mlp_w_0.65	0.65	0.142413								
	↪231796	0.938401	69.678416			0.324324	0.342857				
	↪333333		4.591746								
4	blend_distance_mlp_w_0.50	0.50	0.142495								
	↪232199	0.938151	70.373096			0.324324	0.342857				
	↪333333		4.591746								

6	blend_distance_mlp_w_0.75	0.75	0.142499	0.
	↪231709 0.938456 69.123918	0.324324	0.342857	0.
	↪333333 4.591746			
3	blend_distance_mlp_w_0.35	0.35	0.142812	0.
	↪232928 0.937701 70.322460	0.324324	0.342857	0.
	↪333333 4.591746			
2	blend_distance_mlp_w_0.25	0.25	0.143150	0.
	↪233591 0.937289 70.609525	0.324324	0.342857	0.
	↪333333 4.591746			
7	blend_distance_mlp_w_1.00	1.00	0.143208	0.
	↪232129 0.938201 69.577138	0.324324	0.342857	0.
	↪333333 4.591746			
1	blend_distance_mlp_w_0.15	0.15	0.143602	0.
	↪234395 0.936787 70.893635	0.324324	0.342857	0.
	↪333333 4.591746			
0	blend_distance_mlp_w_0.00	0.00	0.144502	0.
	↪235858 0.935867 71.234496	0.324324	0.342857	0.
	↪333333 4.591746			

	near_zone_MAE_R95	near_zone_R95_bias
5	24.343948	13.508633
4	24.403969	13.620565
6	24.342778	13.434012
3	24.619977	13.732496
2	24.827579	13.807117
7	24.579639	13.247459
1	25.092594	13.881738
0	25.543196	13.993670

	strategy	mlp_weight	mean_normalized_MAE	mean_normalized_RMSE	mean_R2	mean_P95_AE	feasible_precision	feasible_recall	feasible_f1	near_zone_MAE_P80
5	blend_distance_mlp_w_0.65	0.65	0.142413	0.						
	↪231796 0.938401 69.678416	0.324324	0.342857	0.						
	↪333333 4.591746									
6	blend_distance_mlp_w_0.75	0.75	0.142499	0.						
	↪231709 0.938456 69.123918	0.324324	0.342857	0.						
	↪333333 4.591746									
4	blend_distance_mlp_w_0.50	0.50	0.142495	0.						
	↪232199 0.938151 70.373096	0.324324	0.342857	0.						
	↪333333 4.591746									
3	blend_distance_mlp_w_0.35	0.35	0.142812	0.						
	↪232928 0.937701 70.322460	0.324324	0.342857	0.						
	↪333333 4.591746									
7	blend_distance_mlp_w_1.00	1.00	0.143208	0.						
	↪232129 0.938201 69.577138	0.324324	0.342857	0.						
	↪333333 4.591746									

2	blend_distance_mlp_w_0.25	0.25	0.143150	0.
↪233591	0.937289	70.609525	0.324324	0.342857
↪333333	4.591746			
1	blend_distance_mlp_w_0.15	0.15	0.143602	0.
↪234395	0.936787	70.893635	0.324324	0.342857
↪333333	4.591746			
0	blend_distance_mlp_w_0.00	0.00	0.144502	0.
↪235858	0.935867	71.234496	0.324324	0.342857
↪333333	4.591746			

	near_zone_MAE_R95	near_zone_R95_bias	decision_score
5	24.343948	13.508633	0.235131
6	24.342778	13.434012	0.235183
4	24.403969	13.620565	0.235375
3	24.619977	13.732496	0.235983
7	24.579639	13.247459	0.236060
2	24.827579	13.807117	0.236587
1	25.092594	13.881738	0.237360
0	25.543196	13.993670	0.238846

Best MLP weight by conservative decision score: 0.65

1.9 9. Per-target comparison for the selected blend

```
[9]: if RUN_HYBRID_DISTANCE_CV:
    selected_pred = blend_predictions[BEST_MLP_WEIGHT]
    selected_metrics, selected_constraint = evaluate_prediction_frame(
        selected_pred, f"selected_blend_w_{BEST_MLP_WEIGHT:.2f}"
    )

    comparison_for_selected = pd.concat([
        et_metrics.assign(strategy="ExtraTrees_final"),
        selected_metrics,
    ], ignore_index=True)

    display(comparison_for_selected.sort_values(["target", "MAE"])[[
        "strategy", "target", "MAE", "RMSE", "R2", "P95_AE", "Bias",
        ↪"normalized_MAE", "normalized_RMSE"
    ]])
    display(pd.concat([et_constraint, selected_constraint], ignore_index=True))
else:
    print("Run CV first.")
```

	strategy	target	MAE	RMSE	R2	↪
↪P95_AE	Bias	normalized_MAE	normalized_RMSE			
0	ExtraTrees_final	P80	7.653693	10.163284	0.976102	21.
↪564071	-0.003650	0.116397	0.154562			

```

6   selected_blend_w_0.65          P80   7.653693  10.163284  0.976102  21.
↳564071 -0.003650          0.116397      0.154562
10  selected_blend_w_0.65          R50_fines 49.508153  76.997295  0.902712  170.
↳148148 -0.490163          0.200520      0.311858
4   ExtraTrees_final             R50_fines 50.379136  78.486939  0.898911  172.
↳456100 -0.456241          0.204048      0.317891
11  selected_blend_w_0.65          R50_oversize 22.317600  37.833312  0.878230  80.
↳327353 -0.201938          0.205812      0.348897
5   ExtraTrees_final             R50_oversize 22.757123  38.642620  0.872964  81.
↳415772  0.007746          0.209865      0.356360
9   selected_blend_w_0.65          R95   40.236457  65.973277  0.923484  145.
↳924860 -0.081171          0.168676      0.276568
3   ExtraTrees_final             R95   41.418662  68.567830  0.917348  151.
↳864970 -0.447065          0.173632      0.287444
1   ExtraTrees_final             fines_frac 0.006320  0.013829  0.959351  0.
↳031068 -0.000029          0.092129      0.201582
7   selected_blend_w_0.65          fines_frac 0.006320  0.013829  0.959351  0.
↳031068 -0.000029          0.092129      0.201582
2   ExtraTrees_final             oversize_frac 0.025688  0.035234  0.990528  0.
↳074997 -0.000083          0.070944      0.097309
8   selected_blend_w_0.65          oversize_frac 0.025688  0.035234  0.990528  0.
↳074997 -0.000083          0.070944      0.097309

    n_true_feasible  n_pred_feasible  true_positive  false_positive
↳false_negative  feasible_precision  feasible_recall  feasible_f1
↳near_zone_count  near_zone_MAE_P80  \
0           35           37           12           25
↳23           0.324324          0.342857          0.333333          372
↳ 4.591746
1           35           37           12           25
↳23           0.324324          0.342857          0.333333          372
↳ 4.591746

    near_zone_MAE_R95  near_zone_R95_bias          strategy
0           25.543196          13.993670      ExtraTrees_final
1           24.343948          13.508633  selected_blend_w_0.65

```

1.10 10. Optional final training and alternative submission

Only create an alternative test submission if the selected blend is clearly better on OOF validation.

This notebook does **not** overwrite outputs/submissions/prediction_submission.csv.

```

[10]: CREATE_HYBRID_MLP_DISTANCE_SUBMISSION = False

class FinalHybridDistanceModel:
    def __init__(self, mlp_weight=0.0, random_state=42, n_estimators=800):
        self.mlp_weight = float(mlp_weight)

```

```

self.random_state = random_state
self.n_estimators = n_estimators
self.et_pipelines_ = {}
self.mlp_distance_models_ = {}

def fit(self, X_raw, y_df):
    self.et_pipelines_ = {}
    self.mlp_distance_models_ = {}

    # ExtraTrees for all targets
    for i, target in enumerate(target_cols):
        pipe = build_target_et_pipeline(
            target,
            random_state=self.random_state + i,
            n_estimators=self.n_estimators,
        )
        pipe.fit(X_raw, y_df[target])
        self.et_pipelines_[target] = pipe

    # MLP only for distance targets, if weight > 0
    if self.mlp_weight > 0:
        for i, target in enumerate(distance_targets):
            X_target = build_feature_matrix_for_target(X_raw, target)
            mlp = build_mlp(random_state=self.random_state + 100 + i)
            mlp.fit(X_target, y_df[target])
            self.mlp_distance_models_[target] = mlp

    return self

def predict(self, X_raw):
    preds = pd.DataFrame(index=X_raw.index)

    # Start with ExtraTrees for all targets
    for target in target_cols:
        et_pred = self.et_pipelines_[target].predict(X_raw)
        preds[target] = clip_predictions(et_pred, target)

    # Blend MLP only for distances
    if self.mlp_weight > 0:
        for target in distance_targets:
            X_target = build_feature_matrix_for_target(X_raw, target)
            mlp_pred = self.mlp_distance_models_[target].predict(X_target)
            blended = (1 - self.mlp_weight) * preds[target].values + self.
↪ mlp_weight * mlp_pred
            preds[target] = clip_predictions(blended, target)

    return preds[target_cols]

```

```

if CREATE_HYBRID_MLP_DISTANCE_SUBMISSION:
    print("Training final hybrid MLP-distance model with weight:",
    ↪BEST_MLP_WEIGHT)
    hybrid_model = FinalHybridDistanceModel(
        mlp_weight=BEST_MLP_WEIGHT,
        random_state=42,
        n_estimators=800,
    )
    hybrid_model.fit(raw_train, y)
    test_pred = hybrid_model.predict(raw_test)

    submission_hybrid = pd.DataFrame({"scenario_id": np.arange(len(raw_test))})
    for col in target_cols:
        submission_hybrid[col] = test_pred[col].values
    submission_hybrid = submission_hybrid[["scenario_id"] + target_cols]

    hybrid_submission_path = SUBMISSIONS_DIR /
    ↪f"prediction_submission_hybrid_mlp_distance_w_{BEST_MLP_WEIGHT:.2f}.csv"
    hybrid_model_path = MODELS_DIR /
    ↪f"final_hybrid_mlp_distance_w_{BEST_MLP_WEIGHT:.2f}.joblib"
    metadata_path = MODELS_DIR / f"final_hybrid_mlp_distance_w_{BEST_MLP_WEIGHT:
    ↪.2f}_metadata.json"

    submission_hybrid.to_csv(hybrid_submission_path, index=False)
    joblib.dump(hybrid_model, hybrid_model_path)

    metadata = {
        "model_name": "FinalHybridDistanceModel",
        "mlp_weight": BEST_MLP_WEIGHT,
        "target_columns": target_cols,
        "input_columns": input_cols,
        "distance_targets_blended_with_mlp": distance_targets,
        "fragmentation_targets_extra_trees_only": fragmentation_targets,
        "submission_path": str(hybrid_submission_path),
        "model_path": str(hybrid_model_path),
        "constraints": output_constraints,
    }
    with open(metadata_path, "w") as f:
        json.dump(metadata, f, indent=2)

    print("Saved hybrid submission to:", hybrid_submission_path)
    print("Saved hybrid model to:", hybrid_model_path)
    print("Saved metadata to:", metadata_path)
    print("Shape:", submission_hybrid.shape)
    display(submission_hybrid.head())
else:

```

```
print("Alternative hybrid submission creation is disabled by default.")
print("Set CREATE_HYBRID_MLP_DISTANCE_SUBMISSION = True only if the CV_
↪results clearly justify it.")
```

Alternative hybrid submission creation is disabled by default.

Set CREATE_HYBRID_MLP_DISTANCE_SUBMISSION = True only if the CV results clearly justify it.

1.11 11. Decision checklist

Use this checklist before replacing the official forward submission:

1. Does the selected blend improve mean normalized MAE vs ExtraTrees final?
2. Does it improve or at least not degrade RMSE and P95 materially?
3. Does feasible F1 remain equal or improve?
4. Does near-zone R95 MAE improve?
5. Are the gains large enough to justify replacing a stable final model?

For a one-shot submission, prefer the original final model if the gain is only tiny or mixed.